

LO2: Microservice API Anomaly Dataset of Logs and Metrics

Alexander Bakhtin
University of Oulu
Finland
alexander.bakhtin@oulu.fi

Jesse Nyyssölä
University of Helsinki
Finland
jesse.nyyssola@helsinki.fi

Yuqing Wang
University of Helsinki
Finland
yuqing.wang@helsinki.fi

Noman Ahmad
University of Oulu
Finland
noman.ahmad@oulu.fi

Ke Ping
University of Helsinki
Finland
ke.ping@helsinki.fi

Matteo Esposito
University of Oulu
Finland
matteo.esposito@oulu.fi

Mika Mäntylä
University of Helsinki
Finland
mika.mantyla@helsinki.fi

Davide Taibi
University of Oulu
Finland
davide.taibi@oulu.fi

Abstract

Context. Microservice-based systems have gained significant attention over the past years. A critical factor for understanding and analyzing the behavior of these systems is the collection of monitoring data such as logs, metrics, and traces. These data modalities can be used for anomaly detection and root cause analysis of failures. In particular, multi-modal methods utilizing several types of this data at once have gained traction in the research community since these three modalities capture different dimensions of system behavior.

Aim. We provide a dataset that supports research on anomaly detection and architectural degradation in microservice systems. We generate a comprehensive dataset of logs, metrics, and traces from a production microservice system to enable the exploration of multi-modal fusion methods that integrate multiple data modalities.

Method. We dynamically tested the various APIs of the MS-based system, implementing the OAuth2.0 protocol using the Locust tool. For each execution of the prepared test suite, we collect logs and performance metrics for correct and erroneous calls with data labeled according to the error triggered during the call.

Contributions. We collected approximately 657,000 individual log files, totaling over two billion log lines. In addition, we collected more than 45 million individual metric files that contain 485 unique metrics. We provide an initial analysis of logs, identify key metrics through PCA, and discuss challenges in collecting traces for this system. Moreover, we highlight the possibilities for making a more fine-grained version of the data set. This work advances anomaly detection in microservice systems using multiple data sources.

CCS Concepts

• **Computer systems organization** → **Distributed architectures**; • **Security and privacy** → **Intrusion/anomaly detection and malware mitigation**; • **Information systems** → **Test collections**.

Keywords

Microservice Systems, Anomaly Detection, OAuth2.0, Logs, Traces, Metrics, Multi-modal Fusion Methods, Dataset

ACM Reference Format:

Alexander Bakhtin, Jesse Nyyssölä, Yuqing Wang, Noman Ahmad, Ke Ping, Matteo Esposito, Mika Mäntylä, and Davide Taibi. 2025. LO2: Microservice API Anomaly Dataset of Logs and Metrics. In *21st International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '25)*, June 26, 2025, Trondheim, Norway. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3727582.3728682>

1 Introduction

We live in a constantly connected world where Microservice-based systems (MSS) permeate our daily activities. Due to their pervasiveness, industry and researchers are continually investigating multiple aspects, including architectures, patterns, antipatterns, performances, and the evolution of MSS [1, 3, 26, 28].

In MSS research and development, we must execute complex and large systems to collect tracing data to analyze their behavior, gather insights, and test new and emerging ideas. Nonetheless, executing and collecting relevant nontrivial data is a lengthy process that requires considerable effort. Despite the daunting need for public and real-world-like datasets, researchers investigated a few related ones in the literature and executed similar systems every time.

A particular domain where microservice (MS) execution is needed is anomaly detection. A microservice anomaly generally refers to unexpected behaviors or performance issues within a microservices architecture that deviates from normal behavior [27, 34]. Anomalies can manifest as increased latency, resource exhaustion, unexpected failures, or performance degradation, often resulting from complex interactions within the distributed system [31, 32].

These anomalies can be generated through various reliability testing practices. For example, in fault injection testing, the system is introduced with faults such as network issues, hardware exhaustion, system crashes, and configuration errors [24]. Other practices include stress testing and endurance testing, which refer to running the system under a heavy load or long duration, respectively. Current research has placed emphasis on such performance-related anomalies (See Chapter 2).

Traces, logs, and metrics are fundamental for anomaly detection in MSS [31]. Anomaly detection requires the availability of large datasets containing data from different sources, including the log or traces of each service individually or metrics describing the performance of the infrastructure or an individual service container. Traces capture the flow of requests as they move through service instances (i.e., spans) within the MSS. Logs record execution details of service instances. Metrics are quantitative measurements that



This work is licensed under a Creative Commons Attribution 4.0 International License. *PROMISE '25, Trondheim, Norway*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1594-5/2025/06
<https://doi.org/10.1145/3727582.3728682>

track the performance of an entity, such as MS, or the entire host system over time.

Although MSS datasets that include all three modalities have been openly published [18, 29, 31], most of them either provide data on OSS benchmark ([18, 20, 31]) or proprietary, closed systems ([20, 29]). Other datasets do not provide all three modalities [20, 32].

To help researchers merge the gap above, we aimed to provide a dataset of logs, metrics, and traces based on executing commercial and production yet open-sourced MS-based system, Light-OAuth2. In this context, our work contributes to the community by providing:

A dataset of monitoring data for a production-ready (non-demo) MSS. Many existing similar datasets use either benchmark demo systems such as *TrainTicket* or *DeathStarBench*, or commercial proprietary MSS whose code cannot be examined. We leverage the OSS production system LightOAuth2, as such approach allows, on the one hand, to use a "realistic" system, but on the other hand, have its code available for static analysis and anomaly injection to deepen the potential analysis of the system. We set up the data collection process for all three modalities, i.e., **logs**, **metrics**, and **traces**, but in the case of LightOAuth2, we only obtained a significant amount of logs and metrics.

The dataset is prepared and shared according to FAIR principles [9]: Findable, Accessible, Interoperable, and Re-usable.

Scripts and tools to replicate our study. We share a package [5], which provides several scripts and detailed instructions on how to run them, enabling an end-to-end process of deploying the system, executing the tests, gathering the data, and saving it with appropriate structure.

Preliminary analysis. For the logs, we perform anomaly detection with binary labels using different representations of log data and analysis models. Additionally, we analyze how well each of the services' log data performs in detecting each error introduced in the data collection process and provide the F-score for all such combinations. For the metrics, we identify the most significant host metrics by performing Principal Component Analysis. We discuss the issues that arose during the data collection process for the traces.

Ideas for future use of the dataset. We discuss a few applications of the dataset, including anomaly detection in microservices using multi-modal data.

Paper Structure: Section 2 discussed related datasets, Section 3 describes the data collection process while Section 4 describes the collected data structure, and Section 5 provide a preliminary analysis of the data; Section 6 highlights possible future use, and Section 7 explains how the community can contribute to updating the dataset; Section 8 explains how to access the dataset and Section 9 presents the license statement; Section 10 discusses the threats to the validity of our study. Finally, Section 11 concludes this work.

2 Related Work

Our novel dataset improves upon existing uni-modal [18], bi-modal [32] and multi-modal [20, 29, 31] datasets by providing monitoring data of an OSS (as opposed to proprietary [20, 29]) and production (as opposed to benchmark [18, 25, 31, 32]) microservice system.

Yu et al. [31] provide a dataset [14] to evaluate their *Nezha* framework for Root Cause Analysis (RCA). The authors provide the evaluation dataset of logs, metrics, and traces mined from *TrainTicket* and *OnlineBoutique*, part of *DeathStarBench* (DS) family of OSS MSS. The authors inject several performance issues to the container deployment, such as CPU contention, CPU consumption, and Network delay, as well as code defect injections for Java and Python languages to emulate returning errors and exceptions.

Zhang et al. [32] propose the *DeepTraLog* deep learning approach for anomaly detection (AD) using logs and traces of MSS. To evaluate the framework, they provide the *DeepTraLog* dataset [10] of logs and traces mined from the *TrainTicket* MSS. The authors use the main implementation as well as an additional 14 versions of the system with different implemented faults provided by developers of *TrainTicket*.

Lee et al. [18] propose the *Eadro* framework to integrate anomaly detection and root cause analysis from logs, traces, and metrics (referred to in the paper as KPIs) using Deep Learning. The authors share the evaluation dataset of logs, traces, and metrics [17] gathered from *TrainTicket* and *DeathStarBench-SocialNetwork* systems. Three performance anomalies are injected into the systems, namely CPU exhaustion, network delay, and packet loss.

Li et al. [20] propose the *DéjàVu* framework to perform root cause analysis of recurring failures in MSS. The approach is based on metrics only, and authors share the evaluation dataset [22]. The authors evaluate their approach using real and injected faults on *TrainTicket* as well as three proprietary MSS: commercial ISP MSS, commercial bank MSS, and its Oracle database. For the three MSS, performance faults such as CPU exhaustion, network delay, packet loss, database limit, low memory, and pod failure. For the Oracle database, the authors manually labeled real historical failures in different classes.

Researchers from Tsinghua University provided the *AIOps challenge* dataset [29] for the challenge of the same name in 2021. The dataset contains all three modalities: logs, metrics, and traces. The challenge in 2021 was to perform Anomaly Detection on this multi-modal data. The injected anomalies include packet loss, high memory usage, network delay, high disk space usage, CPU exhaustion, and JVM resource exhaustion.

Pham et al. provide the RCAEval framework [25], which includes multimodal data of logs, metrics and traces for three OSS microservice benchmarks. The authors injected performance and network anomalies to the container deployment as well introduced code defects into the source code of the services.

The summary of the related datasets is provided in Table 1.

3 Data Collection

In this section, we describe the studied system, the testing framework, running the system with the tests, and querying and collecting all data.

3.1 The System Under Study: Light-OAuth2

The Light-OAuth2 system¹ is an Open-Source implementation of the OAuth2.0 authorization protocol [12] developed by networknt. It consists of 7 microservices and a MySQL database, although other

¹<https://doc.networknt.com/getting-started/light-oauth2/>

Table 1: Existing MSS Monitoring Datasets

Dataset	MSS	Monitoring data			Anomaly type	Injection method	Goal
		Logs	Metrics	Traces			
Nezha [14, 31]	TrainTicket, DS-OnlineBoutique	✓	✓	✓	Performance (4), Code defects (1)	Performance: injection method not described, Code defects: Customized fault injectors	RCA
DeepTraLog [10, 32]	TrainTicket	✓		✓	Asynchronous interaction (3), Multi-Instance interaction (3), Configuration (4), Code defects (4)	Not described	AD
Eadro [17, 18]	TrainTicket, DS-SocialNetwork	✓	✓	✓	Performance (3)	Chaosblade simulation	AD+RCA
DéjàVu [20, 22]	TrainTicket, Commercial ISP MSS, Commercial bank MSS + Oracle database		✓		Performance (10), Functional anomalies/Fail-stop (3)	Failure injection at random locations with ChaosMesh, manual injection	RCA
AIOPS Challenge 2021 [29]	2xCommercial bank MSS	✓	✓	✓	Performance (7)	Not described	AD, RCA
RCAEval [25]	Train-Ticket, DS-OnlineBoutique, SockShop	✓	✓	✓	Performance (6), Code Defects (5)	Failure injection, Code modification	RCA
Our work	Production OSS MSS	✓	✓		API errors	Negative testing	AD

Table 2: List of Services of Light-OAuth2 System and Implemented Locust Tasks

Service	Function	Tasks (correct)	Tasks (errors)
oauth2-token	OAuth2.0	3	12
oauth2-code	OAuth2.0	2	9
oauth2-service	CRUD	5	8
oauth2-client	CRUD	5	10
oauth2-user	CRUD	6	11
oauth2-refresh-token	CRUD	3	3
oauth2-key	Publ. Keys	0	0
mysqldb	Database	-	-

database options are also supported. The selected project was still under development at the time of selection and test preparation. After the commencement of data collection, the project has been archived². Table 2 shows the list and descriptions of all services.

3.2 Test preparation

The goal of the created test suite is to provide broad coverage of the Light-OAuth2 APIs. In particular, we considered all standard OAuth2.0 flows within this framework as well as the management of entities related to these flows, such as clients, services, and users. Our tests cover the following flows (Grant Types) of the OAuth2.0 protocol:

- Authorization code (without PKCE extension)
- Authorization code (with PKCE extension)
- Client credentials
- Refresh token

Additionally, we implemented Create, Read, Update, and Delete operations for Client, User, and Service registration services.

We excluded the outdated and insecure Resource Owner Password flow³, as well as custom functionalities of Light-OAuth2 implementation, such as mapping of clients to services and management of authentications keys. Neither do we consider the scopes of the issued OAuth2.0 tokens, since we do not simulate an actual system utilizing OAuth2.0.

Tests are prepared and executed with Locust⁴, an Open-Source load-testing tool written in Python that allows the testing RESTful APIs and easily defining complex scenarios mimicking different use cases and business logic of the APIs. Each function calling an API is referred to as *task*. A collection of tasks executed together is called a *test*.

Among the authors, the two with the most relevant experience in API quality and security analysis studied the Light-OAuth2 API documentation to implement the relevant tests in Locust. Moreover, these authors reviewed each others' test suites to ensure compliance with the aforementioned documentation.

In the case of CRUD operations, the correct requests to the API were implemented, as well as all possible errors in the request parameters as described in the documentation. As for the OAuth2.0 flows, both the Light-OAuth2 documentation and the official specification of the OAuth2.0 protocol were used, allowing us to implement these tests in a way that is portable to another OAuth2.0 implementation while avoiding ad-hoc additions of Light-OAuth2.

We split the Locust tasks into two groups - those implementing *correct* requests according to the documentation of each service and those introducing *errors*, aka negative tests [13], in the requests on purpose for each error described in the documentation⁵. For example, the Service Registration MS⁶ contains the endpoint /oauth2/service which accepts GET requests to query an existing service, POST requests to register a new service, PUT requests to update a service, and DELETE requests to delete a service. All return status code 200 in case of a properly formulated request, and the corresponding tasks are labeled as *correct*. In case a provided serviceId does not exist, GET, PUT and DELETE requests will return status code 404 SERVICE_NOT_FOUND; in case a given serviceId already exists in the database, the POST request will return status code 400 SERVICE_ID_EXISTS. All of these four API errors have a corresponding Locust error task, and are labeled as *error* tests in the dataset, with each error having its own label.

Table 2 shows the number of tasks implemented for each service.

3.3 Test execution

To execute the tests and collect the data for future analysis, we perform several *runs* of the test suite on the studied system.

We define a *run* as a full execution of the following procedure:

- (1) deploy the Docker Compose of the system,

²<https://github.com/networknt/light-oauth2>

³<https://auth0.com/docs/get-started/authentication-and-authorization-flow/resource-owner-password-flow>

⁴<https://locust.io>

⁵<https://doc.networknt.com/service/oauth/service/>

⁶<https://doc.networknt.com/service/oauth/service/service/>

- (2) wait for all services and MySQL to become operational,
- (3) run only the *correct locust tasks* for 60s (*correct test*),
- (4) query logs, traces, and metrics,
- (5) for each *error locust task*:
 - (a) run correct + that particular error task for 10s (*error test*),
 - (b) query logs, traces, and metrics,
- (6) stop Docker Compose.

The system was deployed, and tests were executed hourly on our research server as a cronjob in July-August 2024 until the disk space reserved on the server was filled up (540 GB). We did **not attempt fault injection**, such as hardware/host anomalies, network delays or slow disk usage, but instead focused on the anomalies that occur in the system due to erroneous API usage in this work.

We produced a total of 1740 *runs* consisting of 54 *tests* in each run (1 with *correct tasks* only + 53 introducing one *error task* each). Uncompressed raw data occupies about 540 GB.

3.4 Data collection

Container logs are obtained by querying docker logs with the corresponding container ID. We also configured Locust to export its logs during each run, including statements for each query done to the MSS.

Traces are obtained by injecting the jaeger-agent⁷ into the Docker Compose deployment. Metrics are obtained similarly by injecting Prometheus' node exporter⁸ into the deployment. To collect the data, we utilized the API of the respective systems to fetch the data every five seconds and store it into JSON files for the metrics and CSV files for the traces.

Example of collected log, metrics and trace data are available in Listings 1, 2, and 3.

4 Data Description

4.1 Logs

The log files are generated by each container, which corresponds to an individual service. Files are saved for each run and each test. There are approximately 657,000 individual log files for the services (1740 runs * 54 tests * 7 services). The content of the log files comes directly from the logger used in each system at the logging level DEBUG. The service logs contain over **two billion** log lines, as illustrated in Figure 2. In the figure, capital letters refer to an explicit logging level in the log line, while lowercase log types are identified with regular expressions. Note that the figure uses square root scaling on the y-axis.

4.2 Metrics

Prometheus metrics are exported as JSON files. In total, there are 485 metrics, which means that across all runs and tests, there are over 45.5 million individual metric files. This approach provides a granular data structure, but for practical purposes, it is sensible to combine several files into a longer time series. We provide data processing scripts for this purpose [4]. The format of individual

metrics varies as they can have different parameters, but they generally contain timestamp-value pairs that effectively form the time series.

4.3 Traces

Traces are exported as CSV files. Similarly to log files, each combination of run, test, and service has its own CSV file with the following fields: Trace ID, Span ID, Operation Name, Start Time, End Time, and Tags. However, in our current dataset, each trace currently contains only a single span. The Light-OAuth2.0 system only includes requests that make direct calls from the system to the MySQL service, while this system was not implemented initially to provide traceability across sub-operations within services. For example, although a single trace might capture a high-level request for the MySQL service, it did not record finer-grained operations, such as individual POST and GET requests, parameter parsing, authentication checks, or response processing. To achieve such a level of detail, we would need to modify Light-OAuth2.0's source code to manually instrument sub-operations within each service interaction using OpenTelemetry. That requires significant code changes and restructuring, which is infeasible due to our limited resources and the system's archived status. Furthermore, most services do not perform database operations that the tracing agent can catch so the respective CSVs will be empty.

4.4 Metadata

In addition to the data, we also provide the metadata. Each run has a log file containing the response logs made by the Locust tests. These are not part of the Light-OAuth2 system but can help in analyzing the results. For instance, if there is an error but the log message given by the system does not provide much information, the Locust log shows exactly which locust task caused it. It also has timestamps, which can be used as a ground truth of when each request was sent.

In our data repository [4] we provide data appendix that contains statistics of the whole dataset such as file size distributions. Studies have shown that simple predictors such as sequence length can achieve competitive anomaly detection rates on certain datasets [15]. As such, file size distributions too can provide a valuable and computationally efficient predictor in anomaly detection.

5 Preliminary Analysis

Our preliminary analysis will focus on analyzing the software execution through log files. Additionally, we perform Principal Component Analysis of the metrics' time series.

5.1 Log analysis

For the analysis of the log files, we use an existing tool for benchmarking log anomaly detection algorithms called LogLead⁹[21]. It supports several representation methods for the data and machine learning models for anomaly detection. In our case, we want to learn to identify the error tests from the correct ones. To that end,

⁷<https://www.jaegertracing.io/>

⁸<https://prometheus.io/>

⁹<https://github.com/EvoTestOps/LogLead>

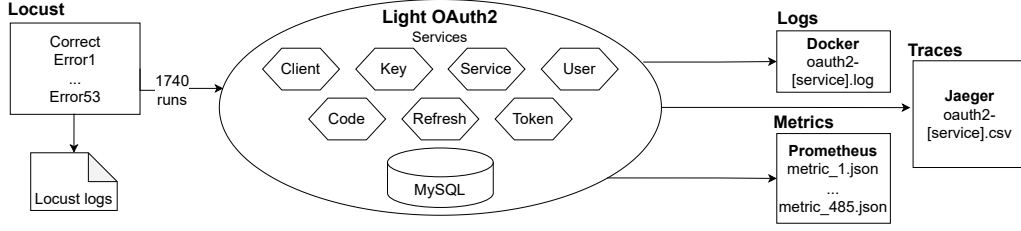


Figure 1: Data Collection Process

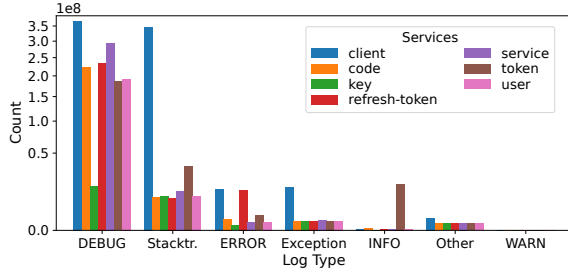


Figure 2: Number of Log Lines per Type and Service

we represent each log file (i.e., a combination of run, test, and service) as their own sequence, and we can label them based on the error tag on the test level.

We perform the log analysis in two stages. In the first stage, we apply anomaly detection models with different representation approaches similar to the previous work [23]. The models included in the approach are DecisionTree (DT), LogisticRegression (LR), IsolationForest (IF), KMeans, RarityModel (RM) and Out-of-Vocabulary Detector (OOVD). DT and LR are the established supervised models from the *scikit-learn* library. IF and KMeans are the unsupervised models from the same library. RM and OOVD are efficient custom unsupervised models introduced in [23]. The unsupervised models work based on the idea that we only use correct training data and measure the deviation. The representations for the log lines are words, trigrams, and parsed event IDs.

The approach in the first stage does not distinguish between individual error types or services. However, the logs from different services vary significantly in content, making it impractical to combine them for analysis by simply appending them. To address this, in the second stage of the analysis, we apply a highly granular anomaly detection approach that analyzes each combination of error cases and service logs individually, producing a 53x7 result matrix. This approach is not practically viable as it requires knowledge of the error cases beforehand, but for the purposes of this paper, it serves as a demonstration of the data and what could be possible. Based on these results, we can propose directions for future work in Section 6.

Both stages use a sample of 100 runs where we select one error test and the correct test from each run. In the first stage, the error is selected randomly for each run, and we utilize the log file of each service, which gives a total of 700 sequences. In the second

stage, only a single service is selected at a time, which results in 200 sequences, but the analysis is done on each service separately. In both stages, the data is evenly divided between normal and anomalous, and it is split randomly with a 50-50 ratio for training and testing. For the first stage, we run the whole setup 10 times and report the means and standard deviations. In the second stage, we run once due to the high number of error-service pairs.

For the supervised models, we use the F1-score metric, which is the standard practice in the field of anomaly detection. For the unsupervised models, we prioritize the AUCROC, which measures the area under the receiver-operating characteristic curve as it measures the performance across all possible thresholds, and the models are very threshold-dependent. For reference, we also provide the AUCROC score for the supervised models and use an optimizer with labels to find the best possible threshold regarding F1-score for the unsupervised models.

The results for the first stage are in Table 3. The asterisk signifies that the unsupervised models used an optimizer for F1-score threshold selection. All of the optimized scores are close to 0.667 which, in practice, means that the model does not work and the optimizer defaults to assigning everything as errors. This leads to the F1-score of 0.667, as shown in the formula below.

$$F_1 = 2 \times \frac{\left(\frac{50}{50+50}\right) \times \left(\frac{50}{50+0}\right)}{\left(\frac{50}{50+50}\right) + \left(\frac{50}{50+0}\right)} = 2 \times \frac{0.5 \times 1.0}{0.5 + 1.0} \approx 0.667$$

The AUCROC scores for the unsupervised models are similarly low at around 0.5, which indicates random classification performance. From all the models only DecisionTree performed reasonably with F1-scores of over 0.7. The representation selection did not have a significant impact. In summary, the results in the first stage suggest that simple concatenation of the services is not a good approach for detecting errors in the dataset.

The whole matrix for the second stage is presented in Table 4. Due to its superior performance, we used the DecisionTree model in the analysis. *Words* were selected as the log representation format as it is computationally the most efficient, and there were no significant performance gains on the other representations.

On average, the Client and Token service logs contain the most relevant content for anomaly detection, while the Key log performs the worst. The performance of other services is highly dependent on the error case being tested, but they can reach high accuracy when used for an appropriate error type. In fact, all other services except for the Key service managed to achieve perfect anomaly

Table 3: Log Analysis Scores by Model and Representation Type

Model	Words		Trigrams		Event IDs	
	F1-score	AUCROC	F1-score	AUCROC	F1-score	AUCROC
IsolationForest	$0.666^* \pm 0.011$	0.489 ± 0.019	$0.666^* \pm 0.007$	0.502 ± 0.018	$0.663^* \pm 0.008$	0.479 ± 0.014
KMeans	$0.674^* \pm 0.011$	0.530 ± 0.018	$0.671^* \pm 0.009$	0.529 ± 0.017	$0.666^* \pm 0.007$	0.516 ± 0.025
RarityModel	$0.666^* \pm 0.011$	0.539 ± 0.011	$0.666^* \pm 0.007$	0.580 ± 0.015	$0.663^* \pm 0.008$	0.511 ± 0.017
OOV Detector	$0.666^* \pm 0.011$	0.416 ± 0.015	$0.661^* \pm 0.012$	0.533 ± 0.012	$0.664^* \pm 0.017$	0.534 ± 0.012
DecisionTree	0.720 ± 0.021	0.731 ± 0.019	0.731 ± 0.025	0.727 ± 0.018	0.700 ± 0.019	0.726 ± 0.018
LogisticReg.	0.467 ± 0.125	0.513 ± 0.020	0.478 ± 0.046	0.500 ± 0.018	0.508 ± 0.066	0.547 ± 0.027

detection accuracy on some error types. This illustrates that having multiple individually trained models can provide comprehensive coverage across all test cases.

The Key service is expected to be the worst predictor because we did not implement Locust tasks for it since its functionality is not part of the standard OAuth2.0 protocol (see Table 2).

The Token service being the best predictor of the anomaly can be explained by the fact that an OAuth flow is a multi-step process, with the Token service being used at the last step, so a lot of successful operations of that service indicate that it happened during the *correct test*, while at any *error test*, Token service would only be invoked correctly a few times due to the errors encountered at the previous steps.

5.2 Metric analysis

Since the exported data contains 485 individual metrics, some containing several time series due to different parameters (see Section 4.2), we perform Principal Component Analysis to identify the most relevant metrics.

From the 172 timestamps and 1124 feature vectors (we consider each metric time series as a separate feature vector but remove Prometheus' metrics), we obtain $\min(172, 1124) = 172$ components in the PCA. We observe that already the Top-3 components explain more than 99% of data variance.

We select Top-5 contributing features from these Top-3 contributing components. We obtain a set of only ten distinct features (out of $\min = 5$ and $\max = 15$): disk written bytes total (for the device used by Docker), filesystem avail. bytes (for the device and filesystem used by Docker), memory AnonHugePages bytes, memory AnonPages bytes, memory Committed AS bytes, memory Inactive anon bytes, memory Inactive bytes, memory MemAvailable bytes, memory MemFree bytes, memory Shmem bytes.

Most of these metrics are related to memory usage during the execution of the tests.

5.3 Traces

Due to the reasons described in Section 4.3, we did not analyze the traces.

6 Future work based on the dataset

As this study focuses on software execution anomalies through negative testing, the log files, in particular, can provide a great dataset for log anomaly detection. Most of the datasets used in log anomaly research consist of logs from monolithic systems, and

even those from distributed systems simply aggregate everything together [33]. The preliminary analysis of LO2 data has shown that it is not possible for our dataset. This raises several challenges:

- If the error is not known, how to select the appropriate service logs for training?
- When running models for multiple services, how to set the threshold and normalize the results between services?
- How to determine the root cause when a single error causes anomalies on multiple services?

Furthermore, the dataset is large enough that each service could be studied separately. In short, the characteristics of the data, as well as its sheer volume, make LO2 a great dataset for future work on log anomaly detection.

While we did not introduce performance anomalies, the dataset provides various metrics that were collected during runtime. These can be used as a basis for time-series analysis by themselves or by fusion with the logs to improve anomaly detection. However, we expect the impact of test case selection to remain light on most performance metrics as the system under study manages to handle the errors gracefully.

The long-term goal for future research is to develop a common schema in order to harmonize varied data formats and structures emanating from traces, logs, and metrics. This schema should permit flexible representation of various architectures while preserving semantic consistency; hence, shared identifiers, such as request IDs and timestamps, are included across all data types to allow their correlation for the exploration of inter-dependencies.

Such a dataset would facilitate future advanced anomaly detection on logs, metrics, and traces in microservice systems, and, in particular, it would support fusion methods relying on multiple modalities. Temporal alignment of data can provide researchers with an overview of system behavior over time and help to detect anomalous patterns that span across sources of data.

7 Contributing to the dataset

We highlight the following possibilities to contribute to this dataset and create an updated, more fine-grained version:

Integrate proper tracing. This can be achieved by either instrumenting the traces in the source code or investigating whether networknt own tracing system¹⁰ integrates with Light-OAuth2.

Change the tasks' spawn rate. In the current version of the dataset, all used Locust tasks (correct and erroneous) had an equal chance of getting executed.

¹⁰<https://doc.networknt.com/architecture/traceability/>

In Locust, it is possible to provide a relative weight to each task, affecting how likely it is to be picked, as well as modifying the amount of spawned users at runtime, making it possible to create more complicated load scenarios more closely representing a production deployment.

It is possible to, e.g., distribute calls with successful results (response 200 or 302) relative to errors (400, 401, 404, 500) by using some known real-life distribution, like one found in [2, 11, 16], or introducing load spikes during the execution of the run.

Capture metrics for each service, and not only host-wide metrics.

Introduce host/hardware anomalies such as slow/breaking connection, slow disk read/write speed, or crashing and redeployment of containers, similar to [18, 20, 31].

Use a different MSS system while following similar test generation, execution, and data collection procedures. This will allow to have a dataset consisting of several MSS for comparison, particularly if it is another implementation of the OAuth2.0 protocol.

Community contributions. We encourage the community to contribute datasets improving upon the aforementioned aspects or providing new systems by opening an issue on our community repository¹¹ and filling in the necessary information. Using GitHub issues allows a public peer review of the candidate dataset. Similarly, issues can be used to request a change or fix a bug in the current dataset or its replication framework.

8 How to use the dataset

The dataset is prepared and shared according to FAIR principles [9] - Findable, Accessible, Interoperable, and Re-usable.

The dataset consists of two repositories shared via Zenodo [4, 5]. One of them consists of the replication package [5] with scripts to deploy the entire system, run the Locust tests, and query the logs, metrics, and traces.

The other one [4] contains the full dataset with 1740 runs as well as some cleaned sample data used for the preliminary analysis in this work. A detailed explanation of the file and data structure is available in the dataset description.

We plan to update the dataset to solve some of the issues of Section 7; so when using the dataset, please cite this paper as well as the exact version of the Zenodo data you used for your study.

9 License

We provide our dataset under the CC BY 4.0 license [6], allowing unrestricted use, distribution, and adaptation, provided that appropriate credit is given to the original authors.

10 Threats to Validity

This section presents the possible threats to the validity of our study, grouped according to Construct, Internal, and External validity, as per Wohlin et al. [30].

Construct Validity refers to how well our measurements reflect our intended assessment. Specific design choices, such as how we measure and filter data, may affect our findings. To address this threat, we relied on previous work [7, 8, 19] when we made such decisions, allowing us to be consistent with what already existed.

Light-OAuth2 has incomplete documentation of its APIs, so the construction of Locust tasks is subject to misunderstanding. However, tests were prepared by two authors who reviewed each other's work. Most CRUD operations operation are standard and cover meaningful error cases that were implemented in our tests, while for OAuth2.0 flows, the official standard of OAuth2.0 [12] serves as an additional source for the construction of Locust tasks, so we could verify that Light-OAuth2.0 implementation follows the standard protocol.

The issues encountered regarding tracing data made it impossible to provide a dataset complete with three modalities (logs, metrics, and traces). The potential solutions to the tracing issue are: (1) use a different system with properly instrumented tracing; (2) manually instrument tracing in the source code of each service of Light-OAuth2; (3) fix and rely on networknt's own implementation of tracing. For the first solution, we are not aware of other easily deployable industrial OSS microservice systems, hence the choice of Light-OAuth2; the other solutions require a great deal of manual effort, which would not represent the actual approach to deploying and tracing microservice at scale adopted in the Industry.

Internal Validity concerns how much the cause-effect relationship in our study can be trusted. Factors that might have threatened these results include a lack of standardization in testing conditions and biased test design. For example, tests run under different conditions, and focusing on specific microservices could warp the dataset. We performed extensive testing of CRUD operations for data-managing services and OAuth2.0 flows like Authorization Code with/without PKCE, Client Credentials, and Refresh Token flow, excluding insecure ones such as Resource Owner Password flow¹². Furthermore, we avoided non-protocol-specific features of the Light-OAuth2 system to ensure portability across OAuth2.0 implementations and running repeated tests on the same machine over an extended period captured load variations, such as during university downtime. We minimized assumptions related to databases by abstracting away from database deployment and using service-managed configurations. Moreover, the database was reset for each test run to prevent performance degradation due to data accumulation. We did not induce hardware anomalies such as network delay or slow disk speed, focusing only on system response to correct and erroneous API use to minimize factors that could be the cause of anomalies in logs or metrics.

External Validity concerns the generalizability of our findings. The dataset, obtained from the MS-based Light-OAuth2 project, is intended to reflect real-world production environments. However, the configurations, choices of databases, and operational loads may limit its applicability beyond our controlled setup. To alleviate these issues, we made sure to use the standard Docker Compose deployment as provided by the authors of the project, only injecting the Jaeger and Prometheus containers, which is standard practice for tracing/metric collection. The choice of the specific database (MySQL) was arbitrary, but it is indeed one of the officially supported databases. The server we could use for the deployment of the system is constrained by resources available at our university and may not represent the capabilities of a real-world production cloud

¹¹<https://github.com/M3SOulu/LO2-Dataset-Community>

¹²<https://auth0.com/docs/get-started/authentication-and-authorization-flow/resource-owner-password-flow>

server. The selection of Locust tasks at runtime selected each task with equal probability, thus potentially providing an unrealistic distribution of correct vs. erroneous calls to the APIs. However, the Locust tool allows easy modification of task weights as well as changes to the distribution of Users at test runtime, so it should be possible to modify the source code of our tests [5] to change the distribution of errors. For example, the distribution of different HTTP error codes can be inferred from datasets such as World Cup 1998 [2, 11, 16]. However, the weights would need to be computed at runtime using reflection based on the selected tasks, so we opted not to resort to it in this work. Despite these constraints, the diversity of test scenarios and the volume of collected data provide a robust foundation for capturing varied system behaviors and error conditions. Additionally, data can be sampled from our dataset with the desired frequencies of correct and error tests without generating a new version of the dataset.

11 Conclusion

In conclusion, the aim of this work was to provide a novel multi-modal dataset of monitoring data consisting of logs, metrics, and traces of a production OSS microservice system.

To achieve this goal, we deployed the Light-OAuth2 MSS and created and executed Locust tests targeting different APIs of the system. We set up the data collection pipeline to gather container logs from Docker, host metrics by injecting Prometheus, and traces by injecting the Jaeger agent. We provided the dataset of logs and metrics, replication instructions, and some data processing pipelines. We described some initial attempts and conclusions of analyzing logs, the most prominent metrics due to PCA, and challenges in gathering traces for this system. This dataset can enable work on anomaly detection in logs and metrics of an MSS, particularly in methods attempting a fusion of log and metric modalities.

Acknowledgement

This work has been funded by the Research Council of Finland (grants n. 359861 and 349488 - MuFAno) and Business Finland (grant 6GSoft).

References

- [1] Roberto Amadini, Simone Gazza, Jacopo Soldani, Monica Vitali, Antonio Brogi, Stefano Forti, Saverio Giallorenzo, Pierluigi Plebani, Francisco Ponce, and Gianluigi Zavattaro. 2024. Pick a Flavour: Towards Sustainable Deployment of Cloud-Edge Applications. In *International Symposium on Logic-Based Program Synthesis and Transformation*. Springer, 117–127.
- [2] M. Arlitt and T. Jin. 2000. A workload characterization study of the 1998 World Cup Web site. *IEEE Network* 14, 3 (2000), 30–37. doi:10.1109/65.844498
- [3] Alexander Bakhtin, Matteo Esposito, Valentina Lenarduzzi, and Davide Taibi. 2025. Network Centrality as a New Perspective on Microservice Architecture. In *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. doi:10.1109/ICSA65012.2025.00017
- [4] Alexander Bakhtin, Jesse Nyssölä, Yuqing Wang, Noman Ahmad, Ke Ping, Matteo Esposito, Mika Mäntylä, and Davide Taibi. 2024. LO2: Microservice Dataset of Logs and Metrics [Data set]. doi:10.5281/zenodo.14257989
- [5] Alexander Bakhtin, Jesse Nyssölä, Yuqing Wang, Noman Ahmad, Ke Ping, Matteo Esposito, Mika Mäntylä, and Davide Taibi. 2024. Replication for "LO2: Microservice Dataset of Logs and Metrics". doi:10.5281/zenodo.14229369
- [6] Creative Commons. 2024. Creative Commons Attribution 4.0 International Public License. <https://creativecommons.org/licenses/by/4.0/> Accessed: 2024-10-29.
- [7] Matteo Esposito and Davide Falessi. 2024. VALIDATE: A deep dive into vulnerability prediction datasets. *Information and Software Technology* (2024), 107448.
- [8] Davide Falessi, Simone Mesiano Laureani, Jonida Çarka, Matteo Esposito, and Daniel Alencar da Costa. 2023. Enhancing the defectiveness prediction of methods and classes via JIT. *Empir. Softw. Eng.* 28, 2 (2023), 37. doi:10.1007/S10664-022-10261-Z
- [9] FORCE11. 2014. The FAIR Data Principles. <https://force11.org/info/the-fair-data-principles/> Accessed: 2024-12-02.
- [10] FudanSELab. 2024. DeepTraLog. <https://github.com/FudanSELab/DeepTraLog>.
- [11] Sona Ghahremani. 2021. 1998 World Cup Website Access Logs. doi:10.5281/zenodo.5145855
- [12] D. Hardt. 2012. The OAuth 2.0 Authorization Framework. Request for Comments 6749. <https://www.rfc-editor.org/rfc/rfc6749>
- [13] Andre Hora. 2024. Test polarity: detecting positive and negative tests. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 537–541.
- [14] IntelligentDDS. 2024. Nezha. <https://github.com/IntelligentDDS/Nezha>.
- [15] Max Landauer, Florian Skopik, and Markus Wurzenberger. 2024. A Critical Review of Common Log Data Sets Used for Evaluation of Sequence-Based Anomaly Detection Techniques. *Proc. ACM Softw. Eng.* 1, FSE, Article 61 (July 2024), 22 pages. doi:10.1145/3660768
- [16] Lawrence Berkeley National Laboratory. 1998. 1998 World Cup Dataset. <https://ita.ee.lbl.gov/html/contrib/WorldCup.html> Accessed: 2024-10-29.
- [17] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R. Lyu. 2023. Eadro. doi:10.5281/zenodo.7615394 [Dataset].
- [18] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R. Lyu. 2023. Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1750–1762.
- [19] Valentina Lenarduzzi, Nyyti Saarimäki, and Davide Taibi. 2019. The Technical Debt Dataset. In *Conference on Predictive Models and Data Analytics in Software Engineering*.
- [20] Zeyan Li, Nengwen Zhao, Mingjie Li, Xianglin Lu, Lixin Wang, Dongdong Chang, Xiaohui Nie, Li Cao, Wenchang Zhang, Kaixin Sui, et al. 2022. Actionable and interpretable fault localization for recurring failures in online service systems. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 996–1008.
- [21] Mika V Mäntylä, Yuqing Wang, and Jesse Nyssölä. 2024. Loglead-fast and integrated log loader, enhancer, and anomaly detector. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 395–399.
- [22] NetManAIops. 2022. DéjàVu. doi:10.5281/zenodo.6955909 [Dataset].
- [23] Jesse Nyssölä and Mika Mäntylä. 2024. Speed and Performance of Parserless and Unsupervised Anomaly Detection Methods on Software Logs. In *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*. 657–666. doi:10.1109/QRS62785.2024.00071
- [24] Joshua Owotogbe, Indika Kumara, Willem-Jan Van Den Heuvel, and Damian Andrew Tamburri. 2024. Chaos Engineering: A Multi-Vocal Literature Review. arXiv:2412.01416 [cs.SE] <https://arxiv.org/abs/2412.01416>
- [25] Luan Pham. 2024. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems. doi:10.5281/ZENODO.14590730
- [26] Simon Schneider and Riccardo Scandariato. 2023. Automatic extraction of security-rich dataflow diagrams for microservice applications written in Java. *Journal of Systems and Software* 202 (2023), 111722.
- [27] Murugiah Souppaya, John Morello, and Karen Scarfone. 2017. *Application Container Security Guide*. Technical Report SP 800-190. National Institute of Standards and Technology (NIST). <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-190.pdf>
- [28] Davide Taibi, Valentina Lenarduzzi, and Claus Pahl. 2017. Processes, motivations, and issues for migrating to microservices architectures: An empirical investigation. *IEEE Cloud Computing* 4, 5 (2017), 22–32.
- [29] Tsinghua University. 2021. AIOps Challenge 2021. <https://www.aiops.cn/gitlab/aiops-nankai/data/trace/aiops2021>.
- [30] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, and Björn Regnell. 2012. *Experimentation in Software Engineering*. Springer.
- [31] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. 2023. Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 553–565.
- [32] Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. 2022. DeeptraLog: Trace-log combined microservice anomaly detection through graph-based deep learning. In *Proceedings of the 44th international conference on software engineering*. 623–634.
- [33] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. 2023. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. 355–366. doi:10.1109/ISSRE59848.2023.00071
- [34] Yuan Zuo, Yulei Wu, Geyong Min, Chengqiang Huang, and Ke Pei. 2020. An Intelligent Anomaly Detection Scheme for Micro-Services Architectures With Temporal and Spatial Data Analysis. *IEEE Transactions on Cognitive Communications and Networking* 6, 2 (2020), 548–561. doi:10.1109/TCCN.2020.2966615

Table 4: Anomaly Detection F1 Score with DecisionTree for a Given Error Test and Service Log

Error Test	Service Log						
	client	code	key	refresh	service	token	user
register_client_400_clientProfile	1.000	0.500	0.227	0.583	0.673	0.981	0.487
register_client_400_clientType	1.000	0.353	0.514	0.871	0.811	0.979	0.644
register_client_404_no_user	1.000	0.305	0.438	0.546	0.846	1.000	0.640
delete_client_404_no_client	1.000	0.472	0.394	0.791	0.822	1.000	0.685
get_client_404_no_client	1.000	0.772	0.372	0.419	0.765	0.959	0.510
get_client_page_400_no_page	1.000	0.328	0.596	0.400	0.746	0.971	0.585
update_client_400_clientProfile	1.000	0.692	0.279	0.583	0.710	0.980	0.529
update_client_400_clientType	1.000	0.723	0.354	0.769	0.706	0.991	0.396
update_client_404_clientId	0.991	0.691	0.606	0.406	0.783	0.990	0.500
update_client_404_ownerId	1.000	0.140	0.455	0.622	0.681	0.991	0.535
authorization_code_client_id_missing_400	0.981	1.000	0.034	0.349	0.783	0.957	0.641
authorization_code_invalid_password_401	0.900	1.000	0.421	0.345	0.932	0.973	0.661
authorization_code_invalid_client_id_404	0.915	0.980	0.282	0.727	0.980	0.980	0.688
authorization_code_missing_response_type_400	0.972	1.000	0.314	0.361	0.813	0.980	0.628
authorization_code_response_not_code_400	0.861	0.981	0.274	0.394	0.869	0.990	0.566
code_challenge_invalid_format_pkce_400	0.969	1.000	0.286	0.418	0.774	1.000	0.560
code_challenge_too_long_pkce_400	0.990	1.000	0.619	0.587	0.819	0.957	0.706
code_challenge_too_short_pkce_400	0.892	0.271	0.074	0.632	0.825	1.000	0.519
code_verifier_missing_pkce_400	0.959	0.451	0.077	0.564	0.805	0.967	0.483
code_verifier_too_long_pkce_400	0.968	0.240	0.100	0.575	0.829	1.000	0.624
code_verifier_too_short_pkce_400	0.950	0.037	0.506	0.537	0.871	0.950	0.596
invalid_code_challenge_method_pkce_400	0.983	1.000	0.328	0.479	0.832	0.957	0.617
invalid_code_verifier_format_PKCE_400	0.920	0.000	0.346	0.308	0.793	0.979	0.509
delete_service_404_no_service	0.945	0.603	0.635	0.692	1.000	1.000	0.577
get_service_404_no_service	0.649	0.709	0.558	0.469	1.000	1.000	0.500
get_service_page_400_no_page	0.917	0.684	0.353	0.442	1.000	0.972	0.505
register_service_400_service_id	1.000	0.175	0.419	0.486	1.000	0.971	0.430
register_service_400_service_type	0.991	0.613	0.080	0.781	1.000	0.963	0.680
register_service_404_no_user	0.980	0.207	0.453	0.448	0.939	0.980	0.420
update_service_404_service_id	0.959	0.196	0.100	0.345	1.000	0.990	0.491
update_service_404_user_id	0.944	0.507	0.619	0.493	1.000	0.952	0.592
access_token_authorization_form_401	0.980	0.419	0.560	0.718	0.943	1.000	0.701
access_token_auth_header_error_401	0.952	0.000	0.518	0.556	0.967	0.991	0.598
access_token_client_id_not_found_404	1.000	0.151	0.185	0.476	0.932	1.000	0.695
access_token_client_secret_wrong_401	0.990	0.118	0.667	0.472	0.929	0.990	0.689
access_token_form_urlencoded_400	0.971	0.419	0.355	0.737	0.961	1.000	0.614
access_token_illegal_grant_type_400	0.848	0.429	0.226	0.689	0.963	1.000	0.611
access_token_missing_authorization_header_400	0.957	0.157	0.513	0.358	0.957	1.000	0.615
delete_token_404	0.970	0.699	0.310	1.000	0.810	0.960	0.531
get_token_404	0.970	0.781	0.347	1.000	0.771	0.974	0.551
get_token_page_400_no_page	0.889	0.271	0.279	1.000	0.752	0.961	0.444
verification_failed_pkce_400	0.991	0.435	0.170	0.324	0.844	0.990	0.638
delete_user_404_no_user	0.970	0.675	0.226	0.640	0.717	0.954	1.000
get_user_404_no_user	0.863	0.138	0.345	0.318	0.757	0.935	1.000
get_user_page_400_no_page	0.967	0.713	0.512	0.281	0.736	0.980	0.981
register_user_400_email_exists	0.979	0.192	0.306	0.508	0.711	0.989	1.000
register_user_400_no_password	0.907	0.271	0.237	0.371	0.825	0.990	1.000
register_user_400_password_no_match	0.871	0.441	0.385	0.535	0.765	1.000	1.000
register_user_400_user_exists	0.973	0.073	0.103	0.508	0.830	0.951	1.000
update_password_400_not_match	0.915	0.318	0.207	0.700	0.729	0.971	1.000
update_password_401_wrong_password	0.969	0.703	0.400	0.375	0.700	0.979	1.000
update_password_404_user_not_found	0.860	0.085	0.250	0.539	0.794	0.981	1.000
update_user_404_no_user	0.957	0.105	0.494	0.387	0.831	0.957	0.991
Average	0.951	0.476	0.353	0.545	0.842	0.980	0.663
Median	0.970	0.435	0.347	0.508	0.825	0.980	0.615
Min	0.649	0.000	0.034	0.281	0.673	0.935	0.396
Max	1.000	1.000	0.667	1.000	1.000	1.000	1.000

Listing 1: Sample of the Collected Log Data

```

16:29:36.735 [XNIO-1 task-1] dfzUR0LpRACkFzgALOYfIA DEBUG c.n.openapi.ApiNormalisedPath <init> - normalised = /oauth2/user
16:29:36.736 [XNIO-1 task-1] dfzUR0LpRACkFzgALOYfIA DEBUG c.n.openapi.ApiNormalisedPath <init> - normalised = /oauth2/user
16:29:36.775 [XNIO-1 task-1] dfzUR0LpRACkFzgALOYfIA INFO com.networkknt.config.Config getConfigStream - Config loaded from default
folder for mask.yml
16:29:36.777 [XNIO-1 task-1] dfzUR0LpRACkFzgALOYfIA DEBUG c.n.openapi.ApiNormalisedPath <init> - normalised = /oauth2/user
16:29:36.846 [XNIO-1 task-1] dfzUR0LpRACkFzgALOYfIA DEBUG com.networkknt.schema.TypeValidator debug - validate( "1", "1", page)
16:29:36.986 [hz._hzInstance_1_dev.cached.thread-16] DEBUG com.zaxxer.hikari.HikariConfig logConfiguration - allowPoolSuspend-
sion...false
16:29:36.987 [hz._hzInstance_1_dev.cached.thread-16] DEBUG com.zaxxer.hikari.HikariConfig logConfiguration - catalog...none
16:29:36.987 [hz._hzInstance_1_dev.cached.thread-16] DEBUG com.zaxxer.hikari.HikariConfig logConfiguration - connectionTest-
Query...none
16:29:36.987 [hz._hzInstance_1_dev.cached.thread-16] DEBUG com.zaxxer.hikari.HikariConfig logConfiguration - dataSource...none
16:29:36.987 [hz._hzInstance_1_dev.cached.thread-16] DEBUG com.zaxxer.hikari.HikariConfig logConfiguration - dataSourceJNDI...none
:

```

Listing 2: Sample of the Collected Metric Data Converted to CSV

timestamp	test_name	gc_duration	gc_duration_count	gc_duration_sum	goroutines	...
1719592192	correct	4.45e-05	8	0.000278365	7	
1719592202	correct	6.0054e-05	10	0.000369997	7	
1719592212	correct	6.0054e-05	13	0.000462525	7	
1719592222	correct	6.0054e-05	16	0.00055404	7	
1719592232	correct	6.0054e-05	18	0.000641278	7	
1719592235	register_client_400_clientType	6.0054e-05	19	0.000670362	7	
1719592245	register_client_400_clientType	6.0054e-05	21	0.000738679	7	
1719592248	register_client_400_clientProfile	6.0054e-05	21	0.000738679	7	
1719592253	register_client_400_clientProfile	6.0054e-05	23	0.000796302	7	
1719592266	register_client_404_no_user	6.0054e-05	26	0.000895424	7	
1719592271	register_client_404_no_user	6.0054e-05	28	0.000947592	7	
1719592274	update_client_400_clientType	6.0054e-05	29	0.000978015	7	
:						

Listing 3: Sample of the Collected Tracing Data

Trace ID	Span ID	Operation Name	Start Time	...
d35bfc89f74f989bc2e291cd2aa0c1eb	b5788b25f6e4db93	SELECT oauth2.refresh_token	2024-06-28T16:30:32.811058+00:00	
19f14689f4137c3487f8466ba1fb0ac4	80d65e80b0665c02	GET	2024-06-28T16:30:32.811603+00:00	
595970b32aea118a69e797ffc909378f	a623b7ba3b40f6b8	INSERT oauth2.refresh_token	2024-06-28T16:30:32.811670+00:00	
859c0ad89a079b2138e47cf9cbad967a	e3da089434118d63	GET	2024-06-28T16:30:32.819811+00:00	
4ef7246eb4a2b37eeba91696df2d0076	c944c05fc1cb2524	GET	2024-06-28T16:30:32.842286+00:00	
1f240bcab152c8298aaf3c94b6ea078f	7048b2701a4e172a	SELECT oauth2.refresh_token	2024-06-28T16:30:32.854248+00:00	
8f69a47b816d6b0679fb76f0a3a80413	7cb1329e9c49492d	INSERT oauth2.refresh_token	2024-06-28T16:30:32.855017+00:00	
4bdde28b2f37b8a3bbd581df74562094	4108c46da844af4d	GET	2024-06-28T16:30:32.869853+00:00	
dc07f0163bcaa297d38663e690421fa0	56f71e99f0de8d81	DELETE oauth2.refresh_token	2024-06-28T16:30:32.870173+00:00	
9aee7bca43b4cb46890c3d859cfff88d	83d608d58133c299	GET	2024-06-28T16:30:32.900300+00:00	
d3b7e505a5acd84ed23a044472948c8d	0edd310418ed7138	SELECT oauth2.refresh_token	2024-06-28T16:30:32.930627+00:00	
4c1c8d9af718143ade073b3366b84b82	df9e38a001f2f0e0	SELECT oauth2.refresh_token	2024-06-28T16:30:32.931304+00:00	
ac725e773cad543871b931a94630614c	491f90da227afd4e	DELETE oauth2.refresh_token	2024-06-28T16:30:32.953960+00:00	
c4db9e752ace6803015728138095ecb3	8643743bdceafd0b	GET	2024-06-28T16:30:32.985495+00:00	
:				